Operating System Practical Slips

Questions + Exam-Oriented Code Only

Prepared from the uploaded T.Y.B.Sc.(C.S.) NEP-2020 Operating System-I practical slips, with repeated questions grouped into reusable program patterns. No sample outputs are included.

1. Banker's Algorithm – Need Matrix and Safe Sequence

Write a C program to simulate Banker's algorithm for deadlock avoidance. Accept Allocation, Max and Available matrices, calculate and display the Need matrix, and check whether the system is in a safe state. If safe, display the safe sequence.

```
#include <stdio.h>

int main()
{
    int n, m, i, j, k;
    int alloc[20][20], max[20][20], need[20][20];
    int avail[20], work[20], finish[20] = {0}, safe[20];
    int count = 0, found;

    printf("Enter number of processes and resources: ");
    scanf("%d%d", &n, &m);

    printf("Enter Allocation matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Max matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("Enter Available vector:\n");
    for(j = 0; j < m; j++) {
        scanf("%d", &avail[j]);
        work[j] = avail[j];
    }

    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    printf("\nNeed Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }

    while(count < n) {
        found = 0;
        for(i = 0; i < n; i++) {
            if(!finish[i]) {
                int possible = 1;
                for(j = 0; j < m; j++)
                    if(need[i][j] > work[j])
                        possible = 0;

                if(possible) {
                    for(j = 0; j < m; j++)
                        work[j] += alloc[i][j];

                    safe[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if(!found)
            break;
    }

    if(count == n) {
        printf("\nSafe Sequence: ");
        for(i = 0; i < n; i++)
            printf("P%d ", safe[i]);
    } else {
        printf("\nSystem is not in safe state.");
```

```
    }

    return 0;
}
```

2. Banker's Resource Request Algorithm

Write a C program to implement the Resource-Request algorithm. Accept the number of processes and resource types, Allocation, Max, Available and a process request. Calculate Need and check whether the request can be granted immediately while keeping the system in a safe state.

```
#include <stdio.h>

int main()
{
    int n, m, i, j, p;
    int alloc[20][20], max[20][20], need[20][20];
    int avail[20], work[20], req[20], finish[20] = {0};
    int safe[20], count = 0, possible, found;

    printf("Enter number of processes and resources: ");
    scanf("%d%d", &n, &m);

    printf("Enter Allocation matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&alloc[i][j]);

    printf("Enter Max matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&max[i][j]);

    printf("Enter Available:\n");
    for(j=0;j<m;j++)
        scanf("%d",&avail[j]);

    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j] = max[i][j] - alloc[i][j];

    printf("Enter process number for request: ");
    scanf("%d",&p);

    printf("Enter request vector:\n");
    for(j=0;j<m;j++)
        scanf("%d",&req[j]);

    for(j=0;j<m;j++) {
        if(req[j] > need[p][j] || req[j] > avail[j]) {
            printf("Request cannot be granted immediately.");
            return 0;
        }
    }

    for(j=0;j<m;j++) {
        avail[j] -= req[j];
        alloc[p][j] += req[j];
        need[p][j] -= req[j];
        work[j] = avail[j];
    }

    while(count < n) {
        found = 0;
        for(i=0;i<n;i++) {
            if(!finish[i]) {
                possible = 1;
                for(j=0;j<m;j++)
                    if(need[i][j] > work[j])
                        possible = 0;

                if(possible) {
                    for(j=0;j<m;j++)
                        work[j] += alloc[i][j];
                    finish[i] = 1;
                    safe[count++] = i;
                    found = 1;
                }
            }
        }
        if(!found) break;
    }

    if(count == n) {
        printf("Request can be granted.");
        printf("\nSafe sequence: ");
        for(i=0;i<n;i++)
            printf("P%d ",safe[i]);
    } else {
        printf("Request cannot be granted safely.");
    }
```

```
    return 0;
}
```

3. FIFO Page Replacement

Write a C program to simulate demand paging using the FIFO page replacement algorithm. Accept the reference string and number of frames and display the page scheduling and total number of page faults.

```
#include <stdio.h>

int main()
{
    int p[50], f[20];
    int n, nf, i, j, pos = 0;
    int hit, fault = 0;

    printf("Enter number of pages: ");
    scanf("%d",&n);

    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);

    printf("Enter number of frames: ");
    scanf("%d",&nf);

    for(i=0;i<nf;i++)
        f[i] = -1;

    for(i=0;i<n;i++) {
        hit = 0;

        for(j=0;j<nf;j++)
            if(f[j] == p[i])
                hit = 1;

        if(!hit) {
            f[pos] = p[i];
            pos = (pos + 1) % nf;
            fault++;
        }

        printf("\n%d : ",p[i]);
        for(j=0;j<nf;j++)
            printf("%d ",f[j]);
    }

    printf("\nPage Faults = %d",fault);
    return 0;
}
```

4. Optimal Page Replacement

Write a C program to simulate demand paging using the Optimal page replacement algorithm. Accept the reference string and number of frames and display the page scheduling and total number of page faults.

```
#include <stdio.h>

int main()
{
    int p[50], f[20];
    int n, nf, i, j, k, pos, far, next;
    int hit, fault = 0;

    printf("Enter number of pages: ");
    scanf("%d",&n);

    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);

    printf("Enter number of frames: ");
    scanf("%d",&nf);

    for(i=0;i<nf;i++)
        f[i] = -1;

    for(i=0;i<n;i++) {
        hit = 0;

        for(j=0;j<nf;j++)
            if(f[j] == p[i])
                hit = 1;

        if(!hit) {
            fault++;
            pos = -1;

            for(j=0;j<nf;j++) {
                if(f[j] == -1) {
                    pos = j;
                    break;
                }
            }

            if(pos == -1) {
                far = -1;
                for(j=0;j<nf;j++) {
                    next = n + 1;
                    for(k=i+1;k<n;k++) {
                        if(f[j] == p[k]) {
                            next = k;
                            break;
                        }
                    }
                    if(next > far) {
                        far = next;
                        pos = j;
                    }
                }
            }

            f[pos] = p[i];
        }

        printf("\n%d : ",p[i]);
        for(j=0;j<nf;j++)
            printf("%d ",f[j]);
    }

    printf("\nPage Faults = %d",fault);
    return 0;
}
```

5. LRU Page Replacement – Counter Method

Write a C program to implement demand paging using the LRU page replacement algorithm using counter method. Accept the reference string and number of frames and display the page scheduling and total number of page faults.

```
#include <stdio.h>

int main()
{
    int p[50], f[20], counter[20];
    int n, nf, i, j, pos, min, hit, fault = 0;
    int time = 0;

    printf("Enter number of pages: ");
    scanf("%d",&n);

    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);

    printf("Enter number of frames: ");
    scanf("%d",&nf);

    for(i=0;i<nf;i++) {
        f[i] = -1;
        counter[i] = 0;
    }

    for(i=0;i<n;i++) {
        time++;
        hit = 0;

        for(j=0;j<nf;j++) {
            if(f[j] == p[i]) {
                hit = 1;
                counter[j] = time;
                break;
            }
        }

        if(!hit) {
            fault++;
            pos = -1;

            for(j=0;j<nf;j++) {
                if(f[j] == -1) {
                    pos = j;
                    break;
                }
            }

            if(pos == -1) {
                min = counter[0];
                pos = 0;
                for(j=1;j<nf;j++) {
                    if(counter[j] < min) {
                        min = counter[j];
                        pos = j;
                    }
                }
            }

            f[pos] = p[i];
            counter[pos] = time;
        }

        printf("\n%d : ",p[i]);
        for(j=0;j<nf;j++)
            printf("%d ",f[j]);
    }

    printf("\nPage Faults = %d",fault);
    return 0;
}
```

6. MFU Page Replacement

Write a C program to simulate demand paging using the MFU page replacement algorithm. Accept the reference string and number of frames and display the page scheduling and total number of page faults.

```
#include <stdio.h>

int main()
{
    int p[50], f[20], freq[20];
    int n, nf, i, j, pos, max, hit, fault = 0;

    printf("Enter number of pages: ");
    scanf("%d",&n);

    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);

    printf("Enter number of frames: ");
    scanf("%d",&nf);

    for(i=0;i<nf;i++) {
        f[i] = -1;
        freq[i] = 0;
    }

    for(i=0;i<n;i++) {
        hit = 0;

        for(j=0;j<nf;j++) {
            if(f[j] == p[i]) {
                freq[j]++;
                hit = 1;
                break;
            }
        }

        if(!hit) {
            fault++;
            pos = -1;

            for(j=0;j<nf;j++) {
                if(f[j] == -1) {
                    pos = j;
                    break;
                }
            }

            if(pos == -1) {
                max = freq[0];
                pos = 0;
                for(j=1;j<nf;j++) {
                    if(freq[j] > max) {
                        max = freq[j];
                        pos = j;
                    }
                }
            }

            f[pos] = p[i];
            freq[pos] = 1;
        }

        printf("\n%d : ",p[i]);
        for(j=0;j<nf;j++)
            printf("%d ",f[j]);
    }

    printf("\nPage Faults = %d",fault);
    return 0;
}
```

7. FCFS CPU Scheduling

Write a C program to simulate FCFS CPU scheduling. Accept arrival time and CPU burst for n processes. Display Gantt chart, turnaround time, waiting time, average waiting time and average turnaround time.

```
#include <stdio.h>

int main()
{
    int n, i, j, time = 0, temp;
    int pid[20], at[20], bt[20], ct[20], tat[20], wt[20];

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++) {
        pid[i] = i+1;
        printf("Enter AT and BT for P%d: ",pid[i]);
        scanf("%d%d",&at[i],&bt[i]);
    }

    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(at[i] > at[j]) {
                temp=at[i]; at[i]=at[j]; at[j]=temp;
                temp=bt[i]; bt[i]=bt[j]; bt[j]=temp;
                temp=pid[i]; pid[i]=pid[j]; pid[j]=temp;
            }

    printf("\nGantt Chart:\n");

    for(i=0;i<n;i++) {
        if(time < at[i])
            time = at[i];

        printf("| P%d ",pid[i]);
        time += bt[i];
        ct[i] = time;
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }
    printf("|\n");

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);

    return 0;
}
```

8. Non-Preemptive SJF Scheduling

Write a C program to simulate Non-preemptive Shortest Job First scheduling. Accept arrival time and CPU burst for n processes and display Gantt chart, turnaround time, waiting time, average waiting time and average turnaround time.

```
#include <stdio.h>

int main()
{
    int n, i, j, time=0, done=0, idx, temp;
    int pid[20], at[20], bt[20], ct[20], tat[20], wt[20], used[20]={0};
    float awt=0, atat=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT and BT for P%d: ",pid[i]);
        scanf("%d%d",&at[i],&bt[i]);
    }

    printf("\nGantt Chart:\n");

    while(done<n) {
        idx=-1;

        for(i=0;i<n;i++)
            if(!used[i] && at[i]<=time)
                if(idx==-1 || bt[i]<bt[idx])
                    idx=i;

        if(idx==-1) {
            time++;
            continue;
        }

        printf("| P%d ",pid[idx]);
        time += bt[idx];
        ct[idx]=time;
        tat[idx]=ct[idx]-at[idx];
        wt[idx]=tat[idx]-bt[idx];
        used[idx]=1;
        done++;
    }
    printf("|\n");

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++) {
        awt += wt[i];
        atat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }

    printf("\nAverage Waiting Time = %.2f",awt/n);
    printf("\nAverage Turnaround Time = %.2f",atat/n);

    return 0;
}
```

9. Preemptive SJF / Shortest Remaining Time First

Write a C program to simulate Preemptive Shortest Job First (SJF) scheduling. Accept arrival time and CPU burst for n processes and calculate completion time, turnaround time, waiting time and averages.

```
#include <stdio.h>

int main()
{
    int n,i,time=0,completed=0,idx,min;
    int pid[20],at[20],bt[20],rem[20],ct[20],tat[20],wt[20];

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT and BT for P%d: ",pid[i]);
        scanf("%d%d",&at[i],&bt[i]);
        rem[i]=bt[i];
    }

    while(completed<n) {
        idx=-1;
        min=100000;

        for(i=0;i<n;i++) {
            if(at[i]<=time && rem[i]>0 && rem[i]<min) {
                min=rem[i];
                idx=i;
            }
        }

        if(idx==-1) {
            time++;
            continue;
        }

        rem[idx]--;
        time++;

        if(rem[idx]==0) {
            ct[idx]=time;
            tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];
            completed++;
        }
    }

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);

    return 0;
}
```

10. Round Robin Scheduling

Write a C program to simulate Round Robin CPU scheduling. Accept arrival time, CPU burst and time quantum. Display Gantt chart, completion time, turnaround time, waiting time, average waiting time and average turnaround time.

```
#include <stdio.h>

int main()
{
    int n,tq,i,j,time=0,completed=0;
    int pid[20],at[20],bt[20],rem[20],ct[20],tat[20],wt[20];
    float awt=0,atat=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT and BT for P%d: ",pid[i]);
        scanf("%d%d",&at[i],&bt[i]);
        rem[i]=bt[i];
    }

    printf("Enter Time Quantum: ");
    scanf("%d",&tq);

    while(completed<n) {
        int executed=0;

        for(i=0;i<n;i++) {
            if(at[i]<=time && rem[i]>0) {
                executed=1;
                printf("| P%d ",pid[i]);

                if(rem[i]>tq) {
                    time += tq;
                    rem[i] -= tq;
                } else {
                    time += rem[i];
                    rem[i]=0;
                    ct[i]=time;
                    tat[i]=ct[i]-at[i];
                    wt[i]=tat[i]-bt[i];
                    completed++;
                }
            }
        }

        if(!executed)
            time++;
    }

    printf("|\n");
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

    for(i=0;i<n;i++) {
        awt += wt[i];
        atat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }

    printf("\nAverage Waiting Time = %.2f",awt/n);
    printf("\nAverage Turnaround Time = %.2f",atat/n);

    return 0;
}
```

11. Non-Preemptive Priority Scheduling

Write a C program to simulate Non-preemptive Priority scheduling. Accept arrival time, CPU burst and priority for n processes. Display Gantt chart, turnaround time, waiting time and averages.

```
#include <stdio.h>

int main()
{
    int n,i,time=0,done=0,idx;
    int pid[20],at[20],bt[20],pr[20],ct[20],tat[20],wt[20],used[20]={0};
    float awt=0,atat=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT, BT and Priority for P%d: ",pid[i]);
        scanf("%d%d%d",&at[i],&bt[i],&pr[i]);
    }

    while(done<n) {
        idx=-1;

        for(i=0;i<n;i++)
            if(!used[i] && at[i]<=time)
                if(idx==-1 || pr[i]<pr[idx])
                    idx=i;

        if(idx==-1) {
            time++;
            continue;
        }

        printf("| P%d ",pid[idx]);
        time += bt[idx];
        ct[idx]=time;
        tat[idx]=ct[idx]-at[idx];
        wt[idx]=tat[idx]-bt[idx];
        used[idx]=1;
        done++;
    }

    printf("|\n");
    printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

    for(i=0;i<n;i++) {
        awt += wt[i];
        atat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
    }

    printf("\nAverage Waiting Time = %.2f",awt/n);
    printf("\nAverage Turnaround Time = %.2f",atat/n);

    return 0;
}
```

12. Preemptive Priority Scheduling

Write a C program to simulate Preemptive Priority scheduling. Accept arrival time, CPU burst and priority for n processes. Calculate completion time, turnaround time, waiting time and averages.

```
#include <stdio.h>

int main()
{
    int n,i,time=0,completed=0,idx;
    int pid[20],at[20],bt[20],pr[20],rem[20],ct[20],tat[20],wt[20];

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT, BT and Priority for P%d: ",pid[i]);
        scanf("%d%d%d",&at[i],&bt[i],&pr[i]);
        rem[i]=bt[i];
    }

    while(completed<n) {
        idx=-1;

        for(i=0;i<n;i++) {
            if(at[i]<=time && rem[i]>0) {
                if(idx==-1 || pr[i]<pr[idx])
                    idx=i;
            }
        }

        if(idx==-1) {
            time++;
            continue;
        }

        rem[idx]--;
        time++;

        if(rem[idx]==0) {
            ct[idx]=time;
            tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];
            completed++;
        }
    }

    printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);

    return 0;
}
```

13. FCFS Disk Scheduling

Write a C program to simulate disk scheduling using FCFS. Accept total number of disk blocks, disk request string and current head position. Display the order of service and total head movements.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n,i,head,total=0;
    int req[100];

    printf("Enter number of disk requests: ");
    scanf("%d",&n);

    printf("Enter request string: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);

    printf("Enter current head position: ");
    scanf("%d",&head);

    printf("\nService order: %d ",head);

    for(i=0;i<n;i++) {
        total += abs(head-req[i]);
        head = req[i];
        printf("%d ",head);
    }

    printf("\nTotal head movements = %d",total);
    return 0;
}
```

14. SSTF Disk Scheduling

Write a C program to simulate disk scheduling using SSTF. Accept disk request string and current head position. Display the order in which requests are served and total head movements.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n,i,j,head,total=0,index,diff,min;
    int req[100],done[100]={0};

    printf("Enter number of disk requests: ");
    scanf("%d",&n);

    printf("Enter request string: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);

    printf("Enter current head position: ");
    scanf("%d",&head);

    printf("\nService order: ");

    for(i=0;i<n;i++) {
        min=100000;
        index=-1;

        for(j=0;j<n;j++) {
            if(!done[j]) {
                diff=abs(head-req[j]);
                if(diff<min) {
                    min=diff;
                    index=j;
                }
            }
        }

        done[index]=1;
        total += abs(head-req[index]);
        head=req[index];
        printf("%d ",head);
    }

    printf("\nTotal head movements = %d",total);
    return 0;
}
```

15. SCAN Disk Scheduling

Write a C program to simulate disk scheduling using SCAN algorithm. Accept disk size, request string, current head position and direction. Display the service order and total head movements.

```
#include <stdio.h>
#include <stdlib.h>

void sort(int a[], int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }
}

int main()
{
    int n,size,head,i,split,total=0;
    int req[100];

    printf("Enter disk size: ");
    scanf("%d",&size);

    printf("Enter number of requests: ");
    scanf("%d",&n);

    printf("Enter request string: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);

    printf("Enter head position: ");
    scanf("%d",&head);

    sort(req,n);

    printf("Enter direction (0=Left, 1=Right): ");
    scanf("%d",&split);

    printf("\nService order: ");

    if(split==1) {
        for(i=0;i<n;i++)
            if(req[i]>=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }

        total += abs(head-(size-1));
        head=size-1;

        for(i=n-1;i>=0;i--)
            if(req[i]<head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
    } else {
        for(i=n-1;i>=0;i--)
            if(req[i]<=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }

        total += abs(head-0);
        head=0;

        for(i=0;i<n;i++)
            if(req[i]>head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
    }

    printf("\nTotal head movements = %d",total);
    return 0;
}
```

16. C-SCAN Disk Scheduling

Write a C program to simulate disk scheduling using C-SCAN. Accept disk size, request string, current head position and direction. Display the service order and total head movements.

```
#include <stdio.h>
#include <stdlib.h>

void sort(int a[], int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }
}

int main()
{
    int n,size,head,direction,i,total=0;
    int req[100];

    printf("Enter disk size: ");
    scanf("%d",&size);

    printf("Enter number of requests: ");
    scanf("%d",&n);

    printf("Enter request string: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);

    printf("Enter head position: ");
    scanf("%d",&head);

    printf("Enter direction (0=Left, 1=Right): ");
    scanf("%d",&direction);

    sort(req,n);

    printf("\nService order: ");

    if(direction==1) {
        for(i=0;i<n;i++)
            if(req[i]>=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }

        total += abs(head-(size-1));
        head=size-1;

        total += size-1;
        head=0;

        for(i=0;i<n;i++)
            if(req[i]<head || req[i]>=0) {
                if(req[i] < size && req[i] < req[0]) {
                    total += abs(head-req[i]);
                    head=req[i];
                    printf("%d ",head);
                }
            }
    } else {
        for(i=n-1;i>=0;i--)
            if(req[i]<=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }

        total += head;
        head=0;

        total += size-1;
        head=size-1;

        for(i=n-1;i>=0;i--)
            if(req[i]>req[n-1]) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
    }
```

```
    printf("\nTotal head movements = %d",total);
    return 0;
}
```

17. LOOK Disk Scheduling

Write a C program to simulate disk scheduling using LOOK algorithm. Accept disk request string, current head position and direction. Display the request order and total head movements.

```
#include <stdio.h>
#include <stdlib.h>

void sort(int a[], int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }
}

int main()
{
    int n,head,direction,i,total=0;
    int req[100];

    printf("Enter number of requests: ");
    scanf("%d",&n);

    printf("Enter request string: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);

    printf("Enter head position: ");
    scanf("%d",&head);

    printf("Enter direction (0=Left, 1=Right): ");
    scanf("%d",&direction);

    sort(req,n);

    printf("\nService order: ");

    if(direction==1) {
        for(i=0;i<n;i++)
            if(req[i]>=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }

        for(i=n-1;i>=0;i--)
            if(req[i]<head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
    } else {
        for(i=n-1;i>=0;i--)
            if(req[i]<=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }

        for(i=0;i<n;i++)
            if(req[i]>head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
    }

    printf("\nTotal head movements = %d",total);
    return 0;
}
```

18. Sequential / Contiguous File Allocation

Write a C menu-driven program to simulate Sequential (Contiguous) file allocation. Assume a disk with n blocks. Randomly mark some blocks as allocated and implement Show Bit Vector, Create New File, Show Directory, Delete File and Exit.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

struct File {
    char name[20];
    int start;
    int length;
};

int main()
{
    int n, bit[100], choice, i, j, start, len, ok;
    char name[20];
    struct File dir[20];
    int count=0;

    printf("Enter number of disk blocks: ");
    scanf("%d",&n);

    srand(time(NULL));
    for(i=0;i<n;i++)
        bit[i] = rand()%2;

    while(1) {
        printf("\n1.Show Bit Vector");
        printf("\n2.Create New File");
        printf("\n3.Show Directory");
        printf("\n4.Delete File");
        printf("\n5.Exit");
        printf("\nEnter choice: ");
        scanf("%d",&choice);

        if(choice==1) {
            for(i=0;i<n;i++) printf("%d ",bit[i]);
        }
        else if(choice==2) {
            printf("Enter file name and length: ");
            scanf("%s%d",name,&len);

            start=-1;
            for(i=0;i<=n-len;i++) {
                ok=1;
                for(j=0;j<len;j++)
                    if(bit[i+j]) ok=0;

                if(ok) {
                    start=i;
                    break;
                }
            }

            if(start==-1) {
                printf("File cannot be allocated.");
            } else {
                for(j=0;j<len;j++)
                    bit[start+j]=1;

                strcpy(dir[count].name,name);
                dir[count].start=start;
                dir[count].length=len;
                count++;
                printf("File allocated.");
            }
        }
        else if(choice==3) {
            printf("\nFile\tStart\tLength\n");
            for(i=0;i<count;i++)
                printf("%s\t%d\t%d\n",
                       dir[i].name,dir[i].start,dir[i].length);
        }
        else if(choice==4) {
            printf("Enter file name: ");
            scanf("%s",name);

            for(i=0;i<count;i++) {
                if(strcmp(dir[i].name,name)==0) {
                    for(j=0;j<dir[i].length;j++)
                        bit[dir[i].start+j]=0;
```

```
                    for(j=i;j<count-1;j++)
                        dir[j]=dir[j+1];

                    count--;
                    printf("File deleted.");
                    break;
                }
            }
        }
        else if(choice==5)
            break;
    }

    return 0;
}
```

19. Linked File Allocation

Write a C menu-driven program to simulate Linked file allocation. Assume a disk with n blocks, randomly mark blocks as allocated, maintain free blocks, and implement Show Bit Vector, Create New File, Show Directory, Delete File and Exit.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

struct File {
    char name[20];
    int blocks[100];
    int count;
};

int main()
{
    int n,bit[100],choice,i,j,b,freeCount;
    char name[20];
    struct File dir[20];
    int count=0;

    printf("Enter number of disk blocks: ");
    scanf("%d",&n);

    srand(time(NULL));
    for(i=0;i<n;i++)
        bit[i]=rand()%2;

    while(1) {
        printf("\n1.Show Bit Vector");
        printf("\n2.Create New File");
        printf("\n3.Show Directory");
        printf("\n4.Delete File");
        printf("\n5.Exit");
        printf("\nEnter choice: ");
        scanf("%d",&choice);

        if(choice==1) {
            for(i=0;i<n;i++) printf("%d ",bit[i]);
        }
        else if(choice==2) {
            printf("Enter file name: ");
            scanf("%s",name);
            printf("Enter number of blocks: ");
            scanf("%d",&b);

            freeCount=0;
            for(i=0;i<n;i++)
                if(bit[i]==0) freeCount++;

            if(freeCount < b) {
                printf("Not enough free blocks.");
            } else {
                strcpy(dir[count].name,name);
                dir[count].count=0;

                for(i=0;i<n && dir[count].count<b;i++) {
                    if(bit[i]==0) {
                        bit[i]=1;
                        dir[count].blocks[dir[count].count++]=i;
                    }
                }
                count++;
                printf("File allocated.");
            }
        }
        else if(choice==3) {
            printf("\nFile\tBlocks\n");
            for(i=0;i<count;i++) {
                printf("%s\t",dir[i].name);
                for(j=0;j<dir[i].count;j++)
                    printf("%d ",dir[i].blocks[j]);
                printf("\n");
            }
        }
        else if(choice==4) {
            printf("Enter file name: ");
            scanf("%s",name);

            for(i=0;i<count;i++) {
                if(strcmp(dir[i].name,name)==0) {
                    for(j=0;j<dir[i].count;j++)
                        bit[dir[i].blocks[j]]=0;
```

```
                for(j=i;j<count-1;j++)
                    dir[j]=dir[j+1];

                count--;
                printf("File deleted.");
                break;
            }
        }
    }
    else if(choice==5)
        break;
}

return 0;
}
```

20. Indexed File Allocation

Write a C menu-driven program to simulate Indexed file allocation. Assume a disk with n blocks, randomly mark some blocks as allocated, maintain free blocks, and implement Show Bit Vector, Create New File, Show Directory, Delete File and Exit.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

struct File {
    char name[20];
    int indexBlock;
    int blocks[100];
    int count;
};

int main()
{
    int n,bit[100],choice,i,j,b,freeCount,indexBlock;
    char name[20];
    struct File dir[20];
    int count=0;

    printf("Enter number of disk blocks: ");
    scanf("%d",&n);

    srand(time(NULL));
    for(i=0;i<n;i++)
        bit[i]=rand()%2;

    while(1) {
        printf("\n1.Show Bit Vector");
        printf("\n2.Create New File");
        printf("\n3.Show Directory");
        printf("\n4.Delete File");
        printf("\n5.Exit");
        printf("\nEnter choice: ");
        scanf("%d",&choice);

        if(choice==1) {
            for(i=0;i<n;i++) printf("%d ",bit[i]);
        }
        else if(choice==2) {
            printf("Enter file name and number of data blocks: ");
            scanf("%s%d",name,&b);

            indexBlock=-1;
            for(i=0;i<n;i++)
                if(bit[i]==0) {
                    indexBlock=i;
                    break;
                }

            if(indexBlock==-1) {
                printf("No free index block.");
                continue;
            }

            freeCount=0;
            for(i=0;i<n;i++)
                if(bit[i]==0 && i!=indexBlock)
                    freeCount++;

            if(freeCount<b) {
                printf("Not enough free blocks.");
                continue;
            }

            bit[indexBlock]=1;
            strcpy(dir[count].name,name);
            dir[count].indexBlock=indexBlock;
            dir[count].count=0;

            for(i=0;i<n && dir[count].count<b;i++) {
                if(bit[i]==0) {
                    bit[i]=1;
                    dir[count].blocks[dir[count].count++]=i;
                }
            }

            count++;
            printf("File allocated.");
        }
        else if(choice==3) {
```

```
            printf("\nFile\tIndex\tData Blocks\n");
            for(i=0;i<count;i++) {
                printf("%s\t%d\t",dir[i].name,dir[i].indexBlock);
                for(j=0;j<dir[i].count;j++)
                    printf("%d ",dir[i].blocks[j]);
                printf("\n");
            }
        }
        else if(choice==4) {
            printf("Enter file name: ");
            scanf("%s",name);

            for(i=0;i<count;i++) {
                if(strcmp(dir[i].name,name)==0) {
                    bit[dir[i].indexBlock]=0;
                    for(j=0;j<dir[i].count;j++)
                        bit[dir[i].blocks[j]]=0;

                    for(j=i;j<count-1;j++)
                        dir[j]=dir[j+1];

                    count--;
                    printf("File deleted.");
                    break;
                }
            }
        }
        else if(choice==5)
            break;
    }

    return 0;
}
```

21. Shell with count Command

Write a C program that behaves like a shell displaying the prompt '$'. Tokenize the command line and execute commands using a child process. Implement count c filename, count w filename and count l filename.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <ctype.h>

int main()
{
    char cmd[100], *args[20];
    int i;

    while(1) {
        printf("$ ");
        fgets(cmd,sizeof(cmd),stdin);
        cmd[strcspn(cmd,"\n")]=0;

        if(strcmp(cmd,"exit")==0)
            break;

        args[0]=strtok(cmd," ");
        i=0;
        while(args[i]!=NULL && i<19)
            args[++i]=strtok(NULL," ");

        if(args[0] && strcmp(args[0],"count")==0 && i>=2) {
            FILE *fp=fopen(args[2],"r");
            int ch, chars=0, words=0, lines=0, inword=0;

            if(fp==NULL) {
                printf("File cannot be opened.\n");
                continue;
            }

            while((ch=fgetc(fp))!=EOF) {
                chars++;
                if(ch=='\n') lines++;

                if(isspace(ch))
                    inword=0;
                else if(!inword) {
                    words++;
                    inword=1;
                }
            }
            fclose(fp);

            if(strcmp(args[1],"c")==0)
                printf("%d\n",chars);
            else if(strcmp(args[1],"w")==0)
                printf("%d\n",words);
            else if(strcmp(args[1],"l")==0)
                printf("%d\n",lines);
        } else {
            pid_t pid=fork();

            if(pid==0) {
                execvp(args[0],args);
                perror("Command failed");
                exit(1);
            } else if(pid>0) {
                wait(NULL);
            }
        }
    }

    return 0;
}
```

22. Shell with search Command

Write a C program that behaves like a shell displaying the prompt '$'. Execute commands using a child process and implement search f filename pattern, search a filename pattern and search c filename pattern.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char cmd[200], *args[20];
    int i;

    while(1) {
        printf("$ ");
        fgets(cmd,sizeof(cmd),stdin);
        cmd[strcspn(cmd,"\n")]=0;

        if(strcmp(cmd,"exit")==0)
            break;

        args[0]=strtok(cmd," ");
        i=0;
        while(args[i]!=NULL && i<19)
            args[++i]=strtok(NULL," ");

        if(args[0] && strcmp(args[0],"search")==0 && i>=3) {
            FILE *fp=fopen(args[2],"r");
            char line[500];
            int count=0, lineNo=0;

            if(fp==NULL) {
                printf("File cannot be opened.\n");
                continue;
            }

            while(fgets(line,sizeof(line),fp)) {
                lineNo++;
                char *p=line;

                while((p=strstr(p,args[3]))!=NULL) {
                    count++;

                    if(strcmp(args[1],"f")==0) {
                        printf("First occurrence at line %d\n",lineNo);
                        fclose(fp);
                        fp=NULL;
                        break;
                    }

                    if(strcmp(args[1],"a")==0)
                        printf("Occurrence at line %d\n",lineNo);

                    p += strlen(args[3]);
                }

                if(fp==NULL) break;
            }

            if(fp) fclose(fp);

            if(strcmp(args[1],"c")==0)
                printf("Occurrences = %d\n",count);
        } else {
            pid_t pid=fork();

            if(pid==0) {
                execvp(args[0],args);
                perror("Command failed");
                exit(1);
            } else if(pid>0)
                wait(NULL);
        }
    }

    return 0;
}
```

23. Orphan Process

Write a C program to illustrate the concept of an orphan process. The parent process should create a child and terminate before the child has finished its task. Use fork(), sleep(), getpid() and getppid().

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    pid_t pid = fork();

    if(pid < 0) {
        perror("fork");
        return 1;
    }

    if(pid == 0) {
        printf("Child PID = %d\n",getpid());
        printf("Initial Parent PID = %d\n",getppid());

        sleep(5);

        printf("Parent PID after parent termination = %d\n",getppid());
    } else {
        printf("Parent PID = %d\n",getpid());
        sleep(1);
        printf("Parent terminating...\n");
        exit(0);
    }

    return 0;
}
```

24. nice() System Call

Write a C program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice().

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    pid = fork();

    if(pid < 0) {
        perror("fork");
        return 1;
    }

    if(pid == 0) {
        printf("Child PID = %d\n",getpid());
        printf("Child priority before nice = %d\n",nice(0));

        if(nice(-5) == -1)
            perror("nice");

        printf("Child priority after nice request = %d\n",nice(0));
    } else {
        wait(NULL);
        printf("Parent completed.\n");
    }

    return 0;
}
```

25. Parent Bubble Sort and Child Insertion Sort

Implement a C program to accept n integers. The main function creates a child process using fork(). The parent sorts the integers using bubble sort and waits for the child using wait(). The child sorts the integers using insertion sort.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

void bubble(int a[], int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=0;j<n-i-1;j++)
            if(a[j]>a[j+1]) {
                t=a[j]; a[j]=a[j+1]; a[j+1]=t;
            }
}

void insertion(int a[], int n)
{
    int i,j,key;
    for(i=1;i<n;i++) {
        key=a[i];
        j=i-1;
        while(j>=0 && a[j]>key) {
            a[j+1]=a[j];
            j--;
        }
        a[j+1]=key;
    }
}

void display(int a[], int n)
{
    int i;
    for(i=0;i<n;i++)
        printf("%d ",a[i]);
    printf("\n");
}

int main()
{
    int n,i;
    int a[50],b[50];
    pid_t pid;

    printf("Enter number of integers: ");
    scanf("%d",&n);

    for(i=0;i<n;i++) {
        scanf("%d",&a[i]);
        b[i]=a[i];
    }

    pid=fork();

    if(pid<0) {
        perror("fork");
        return 1;
    }

    if(pid==0) {
        insertion(b,n);
        printf("Child sorted using insertion sort: ");
        display(b,n);
    } else {
        bubble(a,n);
        printf("Parent sorted using bubble sort: ");
        display(a,n);
        wait(NULL);
    }

    return 0;
}
```

26. execve() with Binary Search

Implement a C program that accepts an integer array. The parent process sorts the array and passes the sorted array to a child process through command line arguments of execve(). The child program uses execve() to perform binary search for a particular item.

```
/* parent.c */
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

void sort(int a[], int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }
}

int main()
{
    int n,i,key;
    int a[50];
    char *args[60];

    printf("Enter number of integers: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    printf("Enter item to search: ");
    scanf("%d",&key);

    sort(a,n);

    args[0]="./child";
    for(i=0;i<n;i++)
        args[i+1]=(char*)malloc(12);

    for(i=0;i<n;i++)
        sprintf(args[i+1],"%d",a[i]);

    args[n+1]=(char*)malloc(12);
    sprintf(args[n+1],"%d",key);
    args[n+2]=NULL;

    if(fork()==0) {
        execve("./child",args,NULL);
        perror("execve");
        exit(1);
    }

    wait(NULL);
    return 0;
}


/* child.c */
#include <stdio.h>
#include <stdlib.h>

int main(int argc,char *argv[])
{
    int n=argc-2, key=atoi(argv[argc-1]);
    int low=0, high=n-1, mid, i;

    while(low<=high) {
        mid=(low+high)/2;

        if(atoi(argv[mid+1])==key) {
            printf("Item found.\n");
            return 0;
        }

        if(atoi(argv[mid+1])<key)
            low=mid+1;
        else
            high=mid-1;
    }

    printf("Item not found.\n");
    return 0;
```

```
}
```

27. Binary Search Using execve() – Alternative Single Source Layout

Write a C program using fork() and execve() in which the parent sorts an integer array and the child executes a new program to perform binary search on the sorted array.

```
/* This is an exam-friendly two-program solution.
   Compile as:
   gcc parent.c -o parent
   gcc child.c -o child

   parent.c */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int n,i,j,t,key;
    int a[50];
    char *args[60];

    scanf("%d",&n);
    for(i=0;i<n;i++) scanf("%d",&a[i]);
    scanf("%d",&key);

    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }

    args[0]="./child";
    for(i=0;i<n;i++) {
        args[i+1]=malloc(12);
        sprintf(args[i+1],"%d",a[i]);
    }
    args[n+1]=malloc(12);
    sprintf(args[n+1],"%d",key);
    args[n+2]=NULL;

    if(fork()==0)
        execve("./child",args,NULL);
    else
        wait(NULL);

    return 0;
}

/* child.c */
#include <stdio.h>
#include <stdlib.h>

int main(int argc,char *argv[])
{
    int n=argc-2, key=atoi(argv[n+1]);
    int low=0, high=n-1, mid;

    while(low<=high) {
        mid=(low+high)/2;

        if(atoi(argv[mid+1])==key) {
            printf("Found");
            return 0;
        }
        else if(atoi(argv[mid+1])<key)
            low=mid+1;
        else
            high=mid-1;
    }

    printf("Not Found");
    return 0;
}
```

28. Sudoku Solver – Backtracking

Write a Python program to solve a 9x9 Sudoku using backtracking. Find an empty cell, check whether a number is safe in its row, column and 3x3 box, place the number and backtrack when necessary.

```
def find_empty(board):
    for i in range(9):
        for j in range(9):
            if board[i][j] == 0:
                return i, j
    return None


def is_safe(board, row, col, num):
    for x in range(9):
        if board[row][x] == num:
            return False

    for x in range(9):
        if board[x][col] == num:
            return False

    start_row = row - row % 3
    start_col = col - col % 3

    for i in range(start_row, start_row + 3):
        for j in range(start_col, start_col + 3):
            if board[i][j] == num:
                return False

    return True


def solve(board):
    empty = find_empty(board)

    if empty is None:
        return True

    row, col = empty

    for num in range(1, 10):
        if is_safe(board, row, col, num):
            board[row][col] = num

            if solve(board):
                return True

            board[row][col] = 0

    return False


board = [
    [5,3,0,0,7,0,0,0,0],
    [6,0,0,1,9,5,0,0,0],
    [0,9,8,0,0,0,0,6,0],
    [8,0,0,0,6,0,0,0,3],
    [4,0,0,8,0,3,0,0,1],
    [7,0,0,0,2,0,0,0,6],
    [0,6,0,0,0,0,2,8,0],
    [0,0,0,4,1,9,0,0,5],
    [0,0,0,0,8,0,0,7,9]
]

solve(board)
```

29. Maze Path – Backtracking

Write a Python program to find a path through a maze using backtracking. Use 2 as the start cell, 3 as the destination, 0 as blocked cell and 1 as an open cell.

```
def is_safe(maze, x, y, visited):
    n = len(maze)
    return (0 <= x < n and 0 <= y < n and
            maze[x][y] != 0 and not visited[x][y])


def solve_maze(maze, x, y, path, visited, solution):
    path.append((x, y))
    visited[x][y] = True

    if maze[x][y] == 3:
        solution.extend(path)
        return True

    dx = [-1, 1, 0, 0]
    dy = [0, 0, -1, 1]

    for i in range(4):
        nx = x + dx[i]
        ny = y + dy[i]

        if is_safe(maze, nx, ny, visited):
            if solve_maze(maze, nx, ny,
                          path, visited, solution):
                return True

    path.pop()
    visited[x][y] = False
    return False


def find_path(maze):
    n = len(maze)
    visited = [[False] * n for _ in range(n)]
    path = []
    solution = []

    for i in range(n):
        for j in range(n):
            if maze[i][j] == 2:
                if solve_maze(maze, i, j,
                              path, visited, solution):
                    return solution
                return []

    return []
```